HIBERNATE INTERVIEW QUESTIONS

TABLE OF CONTENTS

Hibernate Core Concepts

18. Hibernate Performance Optimization
19. save() vs persist() vs update() vs merge()
20. Optimistic vs Pessimistic Locking
21. Hibernate Bytecode Enhancement
22. @BatchSize
23. @Transactional in Hibernate/Spring
24. Transaction Propagation Levels
25. Hibernate Mapping

Spring Data JPA Concepts

1. What is Spring Data JPA?
2. JPA vs Hibernate vs Spring Data JPA
3. What is JpaRepository?
4. CrudRepository vs PagingAndSortingRepository vs JpaRepository
5. Derived Query Methods
6. @Query Annotation
7. JPQL vs Native SQL
8. @Modifying Queries
9. clearAutomatically Meaning
10. Pagination and Sorting

---

1. What is Hibernate?

Hibernate is a Java-based ORM (Object–Relational Mapping) framework that maps Java objects (entities) to relational database tables.

Hibernate allows working with the database using Java objects instead of SQL queries.

Example without Hibernate (JDBC):

```
INSERT INTO user(id, name) VALUES (?, ?);
```

With Hibernate:

```
User u = new User(1, "Nawaz");
session.save(u);
```

Hibernate internally generates SQL statements and communicates with the database.

---

2. Why Do We Use Hibernate?

Key advantages:

1. Eliminates JDBC boilerplate
2. Automatic object–table mapping
3. HQL (Hibernate Query Language)
4. Lazy loading
5. First-level and second-level caching
6. Automatic dirty checking
7. Database independence
8. Manages entity relationships
9. Auto schema generation
10. Built-in transaction integration

---

3. What is SessionFactory? Why is it Heavy?

`SessionFactory` is a heavyweight, immutable, thread-safe factory responsible for creating `Session` objects.

Created only once per application because it performs expensive operations at startup:

- Scanning entity classes
- Building metadata and mapping models
- Setting up database connection pools
- Initializing second-level cache
- Precomputing SQL
- Preparing ORM metamodel

One SessionFactory = One database.

Stored as a Singleton.

---

4. What is Session?

A Hibernate `Session` represents a single unit of work and provides:

- CRUD operations
- Transaction management
- First-level cache
- Dirty checking
- Query execution (HQL, Criteria, SQL)
- Connection handling

Session is lightweight, not thread-safe, and short-lived.

---

5. Design Patterns Used in Hibernate

SessionFactory

- Factory Pattern
- Singleton Pattern
- Flyweight Pattern

Session

- Unit of Work Pattern
- Identity Map Pattern

---

6. Hibernate Entity States

Hibernate defines four entity lifecycle states:

Transient

- Created using `new`
- Not associated with Session
- No database identity

Persistent

- Associated with Session
- Managed by Hibernate
- Stored in first-level cache

Detached

- Session is closed
- Entity still has DB identity
- Hibernate no longer tracks changes

Removed

- Marked for deletion using `delete()`
- Deleted on commit

A full lifecycle example is included earlier.

---

7. get() vs load()

Feature	get()	load()
Fetch type	Eager	Lazy (proxy)
DB hit	Immediately	When accessing fields
If record not found	null	throws ObjectNotFoundException
Use case	Check existence	Obtain reference

---

8. Lazy Loading vs Eager Loading

Lazy Loading

Related entities are loaded only when accessed.

Eager Loading

Related entities are loaded immediately with the parent.

Default behaviors:

- `@ManyToOne`, `@OneToOne` → EAGER
- `@OneToMany`, `@ManyToMany` → LAZY

---

9. N+1 Select Problem

Occurs when Hibernate executes:

- 1 query for parent
- N queries for each child entity

Causes significant performance overhead.

Fixes include:

- JOIN FETCH
- EntityGraph
- Batch fetching
- Second-level cache

---

10. Hibernate Mappings

Hibernate supports various types of entity relationships.

One-to-One

```
@OneToOne
@JoinColumn(name = "profile_id")
private Profile profile;
```

One-to-Many

```
@OneToMany(mappedBy = "user")
private List<Order> orders;
```

Many-to-One

```
@ManyToOne
@JoinColumn(name = "user_id")
private User user;
```

Many-to-Many

```
@ManyToMany
@JoinTable(
    name = "user_roles",
    joinColumns = @JoinColumn(name = "user_id"),
    inverseJoinColumns = @JoinColumn(name = "role_id")
)
private List<Role> roles;
```

mappedBy

Indicates the non-owning side of the relationship.

@JoinColumn

Defines foreign key column.

@JoinTable

Used for mapping many-to-many or link tables.

---

11. Cascade Types

Cascade operations from parent → child:

- PERSIST
- MERGE
- REMOVE
- REFRESH
- DETACH
- ALL

Used to automatically propagate entity state transitions.

---

12. First-Level Cache (Session Cache)

- Mandatory
- Cannot be disabled
- Per-session cache
- Hibernate checks L1 cache before querying DB

---

13. Second-Level Cache (SessionFactory Cache)

- Optional
- Shared across sessions
- Requires provider (EhCache, Hazelcast, etc.)

Workflow:

1. Check L1
2. Check L2
3. Hit DB
4. Store into cache

---

14. Hibernate Caching for Collections

L1 cache: Stores complete collection in session.

L2 cache: Stores only child IDs, not entire objects.

Enable collection caching using:

```
@Cache(usage = CacheConcurrencyStrategy.READ_WRITE)
```

---

15. Lazy vs Extra-Lazy Loading

Lazy Loading

Accessing collection loads the entire collection.

Extra-Lazy Loading

Loads only required data:

- `size()` triggers COUNT query
- `get(index)` loads a single row

---

16. Dirty Checking

Hibernate automatically detects changed fields in persistent entities and issues minimal UPDATE statements.

No explicit `update()` call is necessary.

---

17. flush() vs commit()

Method	Description
flush()	Synchronizes persistence context with DB but does not commit
commit()	Executes flush() + ends transaction

---

18. Hibernate Performance Optimization Techniques

- Enable L2 cache
- Lazy loading
- JOIN FETCH to avoid N+1
- Projection queries (DTO)

- @BatchSize
- Proper indexing
- Avoid unnecessary relationships

---

19. save() vs persist() vs update() vs merge()

Method	Meaning
save()	Inserts new entity; returns ID
persist()	Makes entity persistent; no ID returned
update()	Reattaches existing detached entity
merge()	Copies detached entity state into managed instance; returns managed instance

---

20. Optimistic vs Pessimistic Locking

Optimistic Locking

Uses `@Version` column; no DB locks.

Pessimistic Locking

Uses DB-level row locks (`SELECT FOR UPDATE`).

Lock modes:

- PESSIMISTIC_READ
- PESSIMISTIC_WRITE
- PESSIMISTIC_FORCE_INCREMENT

---

21. Hibernate Bytecode Enhancement

Enables:

- Attribute-level dirty tracking
- Lazy loading for basic fields
- Removal of proxy objects
- Improved performance

---

22. @BatchSize

Loads entities or collections in batches.

```
@BatchSize(size = 10)
```

HIBERNATE TRANSACTION MANAGEMENT

---

23. What is @Transactional in Hibernate/Spring?

`@Transactional` manages:

- Transaction boundaries
- Rollback rules
- Propagation
- Isolation levels

Behavior:

- Opens transaction
- Executes business logic
- Flushes Hibernate session
- Commits or rolls back

---

24. Transaction Propagation Levels

Propagation	Meaning
REQUIRED	Join existing or create new
REQUIRES_NEW	Always creates new transaction
MANDATORY	Must run in existing transaction
SUPPORTS	Runs with or without transaction
NOT_SUPPORTED	Runs without transaction
NEVER	Throw exception if a transaction exists
NESTED	Creates nested transaction

25. Hibernate Mapping Interview Questions to Expect

Typical questions include:

- Difference between unidirectional and bidirectional mapping
- Why mappedBy is required
- How join table works
- How cascading affects relationships
- Orphan removal vs cascade remove
- When to use @JoinColumn vs @JoinTable

SPRING DATA JPA INTERVIEW QUESTIONS

All topics you listed are included.

26. What is Spring Data JPA?

Spring Data JPA simplifies database access by providing:

- Auto-generated CRUD operations
- Derived query methods
- Pagination and sorting
- Custom queries using JPQL/SQL
- Transparent transaction handling
- Integration with Hibernate ORM

It removes the need to write DAO implementations.

---

27. JPA vs Hibernate vs Spring Data JPA

Technology	Description
JPA	Specification (interfaces only)
Hibernate	JPA implementation; full ORM engine
Spring Data JPA	Abstraction over JPA; generates repository implementations

JPA is a standard.

Hibernate is an implementation.

Spring Data JPA is a framework that uses both.

---

28. What is JpaRepository?

`JpaRepository` provides:

- CRUD operations
- Pagination & sorting
- Batch operations
- Flushing
- JPA-specific methods

Built on:

- CrudRepository
- PagingAndSortingRepository

29. CrudRepository vs PagingAndSortingRepository vs JpaRepository

Interface	Features
CrudRepository	Basic CRUD
PagingAndSortingRepository	CRUD + pagination + sorting
JpaRepository	All above + JPA-specific enhancements + batch operations

30. Derived Query Methods

Spring Data JPA auto-generates queries from method names.

Examples:

```
findByEmail(String email)
findByNameAndStatus(String name, String status)
findTop5ByOrderByCreatedAtDesc()
```

Spring parses method names and builds JPQL queries internally.

31. @Query Annotation

Used to define custom queries.

JPQL example:

```
@Query("SELECT u FROM User u WHERE u.email = :email")
User findByEmail(String email);
```

Native SQL example:

```
@Query(value = "SELECT * FROM users WHERE email = ?1", nativeQuery = tr
ue)
User findNative(String email);
```

---

32. JPQL vs Native SQL

Feature	JPQL	Native SQL
Works with	Entities	Tables
Portability	Fully portable	Database-specific
Use case	ORM-friendly	Complex SQL, vendor features

---

33. @Modifying Queries

Used for write operations:

```
@Modifying
@Query("UPDATE User u SET u.status = 'INACTIVE' WHERE u.id = :id")
void deactivate(Long id);
```

Requires a transaction.

---

34. clearAutomatically Meaning

`clearAutomatically = true` clears persistence context after update/delete queries.

Prevents stale data.

Example:

```
@Modifying(clearAutomatically = true)
```

---

35. Pagination and Sorting

Pagination:

```
Page<User> findAll(Pageable pageable);
```

Sorting:

```
List<User> findAll(Sort.by("name"));
```

---

36. Spring Data JPA Transactions

Spring Data repositories are transactional by default.

- Read methods → read-only transactions
- Write methods → wrapped in @Transactional